

Containers

*It was new.
It was singular.
It was simple.
It must succeed!
– H. Nelson*

- Introduction
- **vector**
 - Elements; Range Checking
- **list**
- **map**
- **unordered_map**
- Container Overview
- Advice

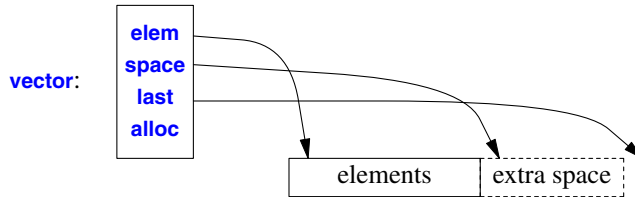
9.1 Introduction

Most computing involves creating collections of values and then manipulating such collections. Reading characters into a **string** and printing out the **string** is a simple example. A class with the main purpose of holding objects is commonly called a *container*. Providing suitable containers for a given task and supporting them with useful fundamental operations are important steps in the construction of any program.

To illustrate the standard-library containers, consider a simple program for keeping names and telephone numbers. This is the kind of program for which different approaches appear “simple and obvious” to people of different backgrounds. The **Entry** class from §8.5 can be used to hold a simple phone book entry. Here, we deliberately ignore many real-world complexities, such as the fact that many phone numbers do not have a simple representation as a 32-bit **int**.

9.2 vector

The most useful standard-library container is **vector**. A **vector** is a sequence of elements of a given type. The elements are stored contiguously in memory. A typical implementation of **vector** (§4.2.2, §4.6) will consist of a handle holding pointers to the first element, one-past-the-last element, and one-past-the-last allocated space (§10.1) (or the equivalent information represented as a pointer plus offsets):



In addition, it holds an allocator (here, **alloc**), from which the **vector** can acquire memory for its elements. The default allocator uses **new** and **delete** to acquire and release memory.

We can initialize a **vector** with a set of values of its element type:

```
vector<Entry> phone_book = {
    {"David Hume",123456},
    {"Karl Popper",234567},
    {"Bertrand Arthur William Russell",345678}
};
```

Elements can be accessed through subscripting:

```
void print_book(const vector<Entry>& book)
{
    for (int i = 0; i!=book.size(); ++i)
        cout << book[i] << '\n';
}
```

As usual, indexing starts at **0** so that **book[0]** holds the entry for **David Hume**. The **vector** member function **size()** gives the number of elements.

The elements of a **vector** constitute a range, so we can use a range-**for** loop (§1.8):

```
void print_book(const vector<Entry>& book)
{
    for (const auto& x : book)    // for "auto" see §1.5
        cout << x << '\n';
}
```

When we define a **vector**, we give it an initial size (initial number of elements):

```
vector<int> v1 = {1, 2, 3, 4};    // size is 4
vector<string> v2;               // size is 0
vector<Shape*> v3(23);           // size is 23; initial element value: nullptr
vector<double> v4(32,9.9);       // size is 32; initial element value: 9.9
```

An explicit size is enclosed in ordinary parentheses, for example, **(23)**, and by default the elements

are initialized to the element type's default value (e.g., `nullptr` for pointers and `0` for numbers). If you don't want the default value, you can specify one as a second argument (e.g., `9.9` for the `32` elements of `v4`).

The initial size can be changed. One of the most useful operations on a `vector` is `push_back()`, which adds a new element at the end of a `vector`, increasing its size by one. For example:

```
void input()
{
    for (Entry e; cin>>e; )
        phone_book.push_back(e);
}
```

This reads `Entry`s from the standard input into `phone_book` until either the end-of-input (e.g., the end of a file) is reached or the input operation encounters a format error.

The standard-library `vector` is implemented so that growing a `vector` by repeated `push_back()`s is efficient. To show how, consider an elaboration of the simple `Vector` from (Chapter 4 and Chapter 5) using the representation indicated in the diagram above:

```
template<typename T>
class Vector {
    T* elem;           // pointer to first element
    T* space;          // pointer to first unused (and uninitialized) slot
    T* last;           // pointer to last slot
public:
    // ...
    int size();         // number of elements (space-elem)
    int capacity();     // number of slots available for elements (last-elem)
    // ...
    void reserve(int newsz); // increase capacity() to newsz
    // ...
    void push_back(const T& t); // copy t into Vector
    void push_back(T&& t);      // move t into Vector
};
```

The standard-library `vector` has members `capacity()`, `reserve()`, and `push_back()`. The `reserve()` is used by users of `vector` and other `vector` members to make room for more elements. It may have to allocate new memory and when it does it moves the elements to the new allocation.

Given `capacity()` and `reserve()`, implementing `push_back()` is trivial:

```
template<typename T>
void Vector<T>::push_back(const T& t)
{
    if (capacity()<size()+1) // make sure we have space for t
        reserve(size()==0?8:2*size()); // double the capacity
    new(space){t};           // initialize *space to t
    ++space;
}
```

Now allocation and relocation of elements happens only infrequently. I used to use `reserve()` to try to improve performance, but that turned out to be a waste of effort: The heuristic used by `vector` is

better than my guesses, so now I only use `reserve()` to avoid relocation of elements when I want to use pointers to elements.

A `vector` can be copied in assignments and initializations. For example:

```
vector<Entry> book2 = phone_book;
```

Copying and moving of `vectors` are implemented by constructors and assignment operators as described in §4.6. Assigning a `vector` involves copying its elements. Thus, after the initialization of `book2`, `book2` and `phone_book` hold separate copies of every `Entry` in the phone book. When a `vector` holds many elements, such innocent-looking assignments and initializations can be expensive. Where copying is undesirable, references or pointers (§1.8) or move operations (§4.6.2) should be used.

The standard-library `vector` is very flexible and efficient. Use it as your default container; that is, use it unless you have a solid reason to use some other container. If your reason is “efficiency,” measure. Our intuition is most fallible in matters of the performance of container uses.

9.2.1 Elements

Like all standard-library containers, `vector` is a container of elements of some type `T`, that is, a `vector<T>`. Just about any type qualifies as an element type: built-in numeric types (such as `char`, `int`, and `double`), user-defined types (such as `string`, `Entry`, `list<int>`, and `Matrix<double,2>`), and pointers (such as `const char*`, `Shape*`, and `double*`). When you insert a new element, its value is copied into the container. For example, when you put an integer with the value `7` into a container, the resulting element really has the value `7`. The element is not a reference or a pointer to some object containing `7`. This makes for nice, compact containers with fast access. For people who care about memory sizes and run-time performance this is critical.

If you have a class hierarchy (§4.5) that relies on `virtual` functions to get polymorphic behavior, do not store objects directly in a container. Instead store a pointer (or a smart pointer; §11.2.1). For example:

```
vector<Shape> vs;           // No, don't - there is no room for a Circle or a Smiley
vector<Shape*> vps;         // better, but see §4.5.4
vector<unique_ptr<Shape>> vups; // OK
```

9.2.2 Range Checking

The standard-library `vector` does not guarantee range checking. For example:

```
void silly(vector<Entry>& book)
{
    int i = book[book.size()].number;    // book.size() is out of range
    // ...
}
```

That initialization is likely to place some random value in `i` rather than giving an error. This is undesirable, and out-of-range errors are a common problem. Consequently, I often use a simple range-checking adaptation of `vector`:

```

template<typename T>
class Vec : public std::vector<T> {
public:
    using vector<T>::vector;           // use the constructors from vector (under the name Vec)

    T& operator[](int i)                // range check
    { return vector<T>::at(i); }

    const T& operator[](int i) const    // range check const objects; §4.2.1
    { return vector<T>::at(i); }
};

```

Vec inherits everything from **vector** except for the subscript operations that it redefines to do range checking. The **at()** operation is a **vector** subscript operation that throws an exception of type **out_of_range** if its argument is out of the **vector**'s range (§3.4.1).

For **Vec**, an out-of-range access will throw an exception that the user can catch. For example:

```

void checked(Vec<Entry>& book)
{
    try {
        book[book.size()] = {"Joe", 999999};    // will throw an exception
        // ...
    }
    catch (out_of_range) {
        cout << "range error\n";
    }
}

```

The exception will be thrown, and then caught (§3.4.1). If the user doesn't catch an exception, the program will terminate in a well-defined manner rather than proceeding or failing in an undefined manner. One way to minimize surprises from uncaught exceptions is to use a **main()** with a **try**-block as its body. For example:

```

int main()
try {
    // your code
}
catch (out_of_range) {
    cerr << "range error\n";
}
catch (...) {
    cerr << "unknown exception thrown\n";
}

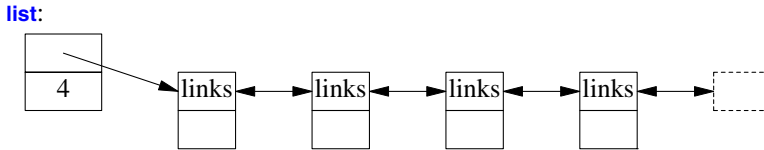
```

This provides default exception handlers so that if we fail to catch some exception, an error message is printed on the standard error-diagnostic output stream **cerr** (§8.2).

Some implementations save you the bother of defining **Vec** (or equivalent) by providing a range-checked version of **vector** (e.g., as a compiler option).

9.3 list

The standard library offers a doubly-linked list called **list**:



We use a **list** for sequences where we want to insert and delete elements without moving other elements. Insertion and deletion of phone book entries could be common, so a **list** could be appropriate for representing a simple phone book. For example:

```
list<Entry> phone_book = {
    {"David Hume",123456},
    {"Karl Popper",234567},
    {"Bertrand Arthur William Russell",345678}
};
```

When we use a linked list, we tend not to access elements using subscripting the way we commonly do for vectors. Instead, we might search the list looking for an element with a given value. To do this, we take advantage of the fact that a **list** is a sequence as described in Chapter 10:

```
int get_number(const string& s)
{
    for (const auto& x : phone_book)
        if (x.name==s)
            return x.number;
    return 0; // use 0 to represent "number not found"
}
```

The search for **s** starts at the beginning of the list and proceeds until **s** is found or the end of **phone_book** is reached.

Sometimes, we need to identify an element in a **list**. For example, we may want to delete it or insert a new entry before it. To do that we use an *iterator*: a **list** iterator identifies an element of a **list** and can be used to iterate through a **list** (hence its name). Every standard-library container provides the functions **begin()** and **end()**, which return an iterator to the first and to one-past-the-last element, respectively (Chapter 10). Using iterators explicitly, we can – less elegantly – write the **get_number()** function like this:

```
int get_number(const string& s)
{
    for (auto p = phone_book.begin(); p!=phone_book.end(); ++p)
        if (p->name==s)
            return p->number;
    return 0; // use 0 to represent "number not found"
}
```

In fact, this is roughly the way the terser and less error-prone range-**for** loop is implemented by the

compiler. Given an iterator **p**, ***p** is the element to which it refers, **++p** advances **p** to refer to the next element, and when **p** refers to a class with a member **m**, then **p->m** is equivalent to **(*p).m**.

Adding elements to a **list** and removing elements from a **list** is easy:

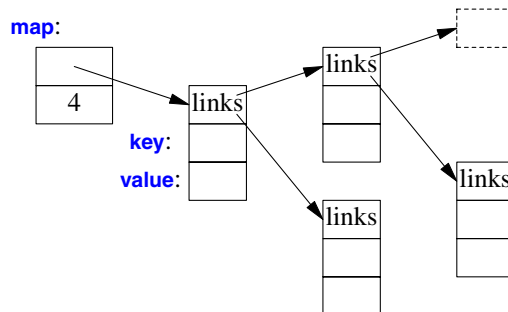
```
void f(const Entry& ee, list<Entry>::iterator p, list<Entry>::iterator q)
{
    phone_book.insert(p,ee);    // add ee before the element referred to by p
    phone_book.erase(q);        // remove the element referred to by q
}
```

For a **list**, **insert(p,elem)** inserts an element with a copy of the value **elem** before the element pointed to by **p**. Similarly, **erase(p)** removes the element pointed to by **p** and destroys it. In both cases, **p** may be an iterator pointing one-beyond-the-end of the **List**.

These **list** examples could be written identically using **vector** and (surprisingly, unless you understand machine architecture) perform better with a small **vector** than with a small **list**. When all we want is a sequence of elements, we have a choice between using a **vector** and a **list**. Unless you have a reason not to, use a **vector**. A **vector** performs better for traversal (e.g., **find()** and **count()**) and for sorting and searching (e.g., **sort()** and **binary_search()**).

9.4 map

Writing code to look up a name in a list of *(name,number)* pairs is quite tedious. In addition, a linear search is inefficient for all but the shortest lists. The standard library offers a search tree (a red-black tree) called **map**:



In other contexts, a **map** is known as an associative array or a dictionary. It is implemented as a balanced binary tree.

The standard-library **map** is a container of pairs of values optimized for lookup. We can use the same initializer as for **vector** and **list** (§9.2, §9.3):

```
map<string,int> phone_book {
    {"David Hume",123456},
    {"Karl Popper",234567},
    {"Bertrand Arthur William Russell",345678}
};
```

When indexed by a value of its first type (called the *key*), a **map** returns the corresponding value of the second type (called the *value* or the *mapped type*). For example:

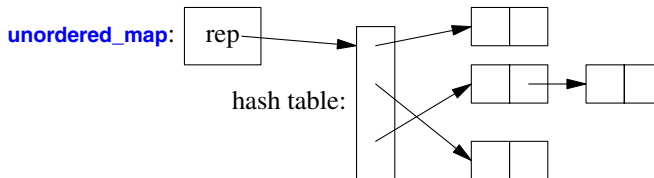
```
int get_number(const string& s)
{
    return phone_book[s];
}
```

In other words, subscripting a **map** is essentially the lookup we called `get_number()`. If a **key** isn't found, it is entered into the **map** with a default value for its **value**. The default value for an integer type is `0`; the value I just happened to choose represents an invalid telephone number.

If we wanted to avoid entering invalid numbers into our phone book, we could use `find()` and `insert()` instead of `[]`.

9.5 unordered_map

The cost of a **map** lookup is $O(\log(n))$ where n is the number of elements in the **map**. That's pretty good. For example, for a **map** with 1,000,000 elements, we perform only about 20 comparisons and indirections to find an element. However, in many cases, we can do better by using a hashed lookup rather than comparison using an ordering function, such as `<`. The standard-library hashed containers are referred to as “unordered” because they don't require an ordering function:



For example, we can use an **unordered_map** from `<unordered_map>` for our phone book:

```
unordered_map<string,int> phone_book {
    {"David Hume",123456},
    {"Karl Popper",234567},
    {"Bertrand Arthur William Russell",345678}
};
```

As for a **map**, we can subscript an **unordered_map**:

```
int get_number(const string& s)
{
    return phone_book[s];
}
```

The standard-library provides a default hash function for **strings** as well as for other built-in and standard-library types. If necessary, you can provide your own. Possibly, the most common need for a “custom” hash function comes when we want an unordered container of one of our own types. A hash function is often provided as a function object (§5.5). For example:


```
struct Record {
    string name;
    int product_code;
    // ...
};

struct Rhash {           // a hash function for Record
    size_t operator()(const Record& r) const
    {
        return hash<string>()(r.name) ^ hash<int>()(r.product_code);
    }
};

unordered_set<Record,Rhash> my_set; // set of Records using Rhash for lookup
```

Creating a new hash function by combining existing hash functions using exclusive or (^) is simple and often very effective.

9.6 Container Overview

The standard library provides some of the most general and useful container types to allow the programmer to select a container that best serves the needs of an application:

Standard Container Summary	
<code>vector<T></code>	A variable-size vector (§9.2)
<code>list<T></code>	A doubly-linked list (§9.3)
<code>forward_list<T></code>	A singly-linked list
<code>deque<T></code>	A double-ended queue
<code>set<T></code>	A set (a <code>map</code> with just a key and no value)
<code>multiset<T></code>	A set in which a value can occur many times
<code>map<K,V></code>	An associative array (§9.4)
<code>multimap<K,V></code>	A map in which a key can occur many times
<code>unordered_map<K,V></code>	A map using a hashed lookup (§9.5)
<code>unordered_multimap<K,V></code>	A multimap using a hashed lookup
<code>unordered_set<T></code>	A set using a hashed lookup
<code>unordered_multiset<T></code>	A multiset using a hashed lookup

The unordered containers are optimized for lookup with a key (often a string); in other words, they are implemented using hash tables.

The containers are defined in namespace `std` and presented in headers `<vector>`, `<list>`, `<map>`, etc. (§6.3). In addition, the standard library provides container adaptors `queue<T>`, `stack<T>`, and `priority_queue<T>`. Look them up if you need them. The standard library also provides more specialized container-like types, such as a fixed-size array `array<T,N>` (§11.3.1) and `bitset<N>` (§11.3.2).

The standard containers and their basic operations are designed to be similar from a notational point of view. Furthermore, the meanings of the operations are equivalent for the various

containers. Basic operations apply to every kind of container for which they make sense and can be efficiently implemented. For example:

- `begin()` and `end()` give iterators to the first and one-beyond-the-last elements, respectively.
- `push_back()` can be used (efficiently) to add elements to the end of a `vector`, `list`, and other containers.
- `size()` returns the number of elements.

This notational and semantic uniformity enables programmers to provide new container types that can be used in a very similar manner to the standard ones. The range-checked vector, `Vector` (§3.4.2, Chapter 4), is an example of that. The uniformity of container interfaces allows us to specify algorithms independently of individual container types. However, each has strengths and weaknesses. For example, subscripting and traversing a `vector` is cheap and easy. On the other hand, `vector` elements are moved when we insert or remove elements; `list` has exactly the opposite properties. Please note that a `vector` is usually more efficient than a `list` for short sequences of small elements (even for `insert()` and `erase()`). I recommend the standard-library `vector` as the default type for sequences of elements: you need a reason to choose another.

Consider the singly-linked list, `forward_list`, a container optimized for the empty sequence (which occupies just one word) because the number of elements are zero or very low; such sequences are surprisingly useful.

9.7 Advice

- [1] The material in this chapter roughly corresponds to what is described in much greater detail in Chapter 31 of [Stroustrup,2013].
- [2] An STL container defines a sequence; §9.2.
- [3] STL containers are resource handles; §9.2, §9.3, §9.4, §9.5.
- [4] Use `vector` as your default container; §9.2, §9.6.
- [5] For simple traversals of a container, use a range-`for` loop or a `begin/end` pair of iterators; §9.2, §9.3.
- [6] Use `reserve()` to avoid invalidating pointers and iterators to elements; §9.2.
- [7] Don't assume performance benefits from `reserve()` without measurement; §9.2.
- [8] Use `push_back()` or `resize()` on a container rather than `realloc()` on an array; §9.2.
- [9] Don't use iterators into a resized `vector`; §9.2.
- [10] Do not assume that `[]` range checks; §9.2.
- [11] Use `at()` when you need guaranteed range checks; §9.2.
- [12] Elements are copied into a container; §9.2.1.
- [13] To preserve polymorphic behavior of elements, store pointers; §9.2.1.
- [14] Insertion operators, such as `insert()` and `push_back()` are often surprisingly efficient on a `vector`; §9.3.
- [15] Use `forward_list` for sequences that are usually empty; §9.6.
- [16] When it comes to performance, don't trust your intuition: measure; §9.2.
- [17] A `map` is usually implemented as a red-black tree; §9.4.
- [18] An `unordered_map` is a hash table; §9.5.

- [19] Pass a container by reference and return a container by value; §9.2.
- [20] For a container, use the `()`-initializer syntax for sizes and the `{}`-initializer syntax for lists of elements; §4.2.3, §9.2.
- [21] Prefer compact and contiguous data structures; §9.3.
- [22] A `list` is relatively expensive to traverse; §9.3.
- [23] Use unordered containers if you need fast lookup for large amounts of data; §9.5.
- [24] Use ordered associative containers (e.g., `map` and `set`) if you need to iterate over their elements in order; §9.4.
- [25] Use unordered containers for element types with no natural order (e.g., no reasonable `<`); §9.4.
- [26] Experiment to check that you have an acceptable hash function; §9.5.
- [27] Hash function obtained by combining standard hash functions for elements using exclusive or are often good; §9.5.
- [28] Know your standard-library containers and prefer them to hand-crafted data structures; §9.6.